

CONFIDENTIAL — FOR DISCUSSION

Connector OS

Infrastructure for Governed AI



A Brief for Greylock Partners

GlobalSushrut, Inc. • 2026

This document answers the questions raised by our buyer one-pager and positions Connector OS as infrastructure, not a product feature. It is written with Greylock's stated investment thesis in mind: *Visibility, Governance, and Auditability* for AI — the same three pillars that define Connector's architecture.

Contents

1	The Greylock Lens: Why This Brief Is Written This Way	3
1.1	What Greylock Has Learned from Cloud Security	3
1.2	What Greylock Specifically Calls Out as Missing	3
2	The Problem: Three Structural Gaps in Every AI Deployment	5
2.1	Gap 1 — Silent Budget Drain	5
2.2	Gap 2 — The Compliance Nightmare	5
2.3	Gap 3 — Blind Trust Is Not a Strategy	6
3	The Architecture: How Connector OS Works	7
3.1	The Three-Stage Pipeline	7
3.1.1	Stage 1 — Isolate (Agent Vaults)	7
3.1.2	Stage 2 — Govern (The Safety Net)	7
3.1.3	Stage 3 — Verify (The Proof Plane)	7
3.2	CCL Contracts — Policy as Compiled Code	8
4	Infrastructure Positioning — Crystal Clear	10
4.1	What Connector IS — and What It Is NOT	10
4.2	Where Connector Sits in the Stack	10
4.3	Why LLM Providers Won't Build This	11
4.4	Why Orchestration Frameworks Won't Build This	11
4.5	Why Cloud Providers Won't Build This	11
5	Answering the Buyer One-Pager	12
6	Moat and Defensibility	14
6.1	Technical Moat — The 9-Chain Architecture	14
6.2	Compiler Moat — CCL as Policy-as-Code	14
6.3	Compliance Lock-In	15
6.4	Network Effect	15
7	Why Now — The Regulatory and Market Forcing Functions	16
7.1	The Regulatory Wave Is Landing	16

7.2	The Agentic AI Explosion	16
7.3	The Cloud Security Timing Parallel	16
8	GTM — The Entry Wedge and Revenue Architecture	17
8.1	The Wedge: Regulated Industries with AI in Production	17
8.2	Revenue Architecture	17
8.3	The Pilot Is the Sales Motion	17
9	Competitive Landscape	18
10	The Ask — What We Are Looking For from Greylock	19
10.1	What We Want to Explore with Greylock	19
10.2	Why This Is Not a Feature — One Final Time	19

1 The Greylock Lens: Why This Brief Is Written This Way

Greylock has publicly articulated the most precise framing of the AI security problem we have seen from any investor. Jason Risch and Saam Motamedi wrote in *Securing AI*:

Greylock — Securing AI (verbatim)

*“The immediate priority CISOs face is **visibility, governance, and auditability** to mitigate risk. . . AI represents a new and currently unsecured surface area (much as cloud did in a previous platform shift). . . As AI agents enter the enterprise, interact with systems, and are potentially granted permissions to modify state, they become a **new identity surface area** to govern.”*

Those three words — Visibility, Governance, Auditability — are the exact three structural properties that Connector OS produces for every agent execution. This is not a coincidence of language. Greylock has identified the same structural gap that Connector’s architecture is designed to close.

1.1 What Greylock Has Learned from Cloud Security

The Greylock thesis draws an explicit parallel between AI security today and cloud security in 2012–2015:

- The infrastructure layers need to stabilize before the right durable architecture emerges.
- Many of the largest cloud security companies launched *years after* the cloud migration began — because enterprises only face the security problem when they are already deep in production.
- The first durable products solve posture (CSPM) and runtime (D&R) — in that order.

Enterprises are already deep in AI production in 2026. They have already deployed agents. The posture problem has landed. The runtime problem is here now. **Connector OS** is the runtime control plane.

1.2 What Greylock Specifically Calls Out as Missing

From the same Greylock research, the five unsolved gaps are:

Gap Greylock Identified	Connector OS Structural Answer
Prompt & response audit logs	Chain 1 (HMAC Audit Chain) — every input/output tamper-evident
PII leaking into LLM context	Chain 9 (Isolation Chain) + namespace firewall — structurally blocked
Agent identity as new surface	Agent DID + namespace isolation + CCL contract capability declarations
Non-determinism / unpredictability	CCL contracts compile policy; Chain 3 (Dehallucination) grounds every output
Runtime detection & response	Policy engine + HITL gate + governance chain — blocks in real time

2 The Problem: Three Structural Gaps in Every AI Deployment

Every enterprise deploying AI agents in production faces three structural gaps that no current tool closes.

2.1 Gap 1 — Silent Budget Drain

The Failure Mode

An agent enters a retry loop at 3 AM. No human is watching. By morning, \$18,000 in LLM API spend has been consumed on a single broken workflow. Nobody knew. Nobody could have known.

The root cause is architectural: AI agents have no native concept of budget. The LLM provider charges per token. The orchestration framework passes requests through. There is no layer between the agent and the provider that enforces a spending limit, counts tokens in real time, and terminates the agent when the budget is exhausted.

Current workarounds that don't work:

- Cloud billing alerts fire 24–48 hours after the fact, by which point the damage is done.
- Application-level counters break under retries, multi-agent delegation, and parallel execution.
- LLM providers offer no per-agent, per-workflow budget enforcement — only account-level limits.

What Connector does: The gateway service intercepts every LLM call. Token counts are computed from the provider's actual response. A per-agent cost ledger is updated in real time. CCL contracts declare a `budget.tokens` ceiling. If the ceiling is hit, the contract terminates — not the application, the *contract* — with a signed receipt proving why it stopped.

2.2 Gap 2 — The Compliance Nightmare

The Failure Mode

A healthcare AI agent reads a patient chart to generate a care recommendation. The agent's context window now contains PHI. That context is sent to the LLM provider. You have just violated HIPAA §164.502(b) minimum necessary and §164.312(a) access controls — with no proof that you prevented it or that it happened.

Why regulations are structurally violated by default:

- LLM context construction is opaque — developers do not inspect what the agent injected.
- Orchestration frameworks (LangChain, CrewAI, AutoGen) have no data classification

layer.

- Compliance is enforced by policy documents, not by the runtime system.

What Connector does: The namespace firewall classifies data at the storage layer. PHI lives in `/p/` namespace with `SecurityLevel::Restricted`. The selective context engine strips or pseudonymizes `/p/` content before it reaches the LLM. Chain 9 (Isolation Chain) records every access attempt — authorized and unauthorized — in a tamper-evident log. A single API call produces a cryptographic proof bundle covering HIPAA, SOC 2 (all five TSC), GDPR, and EU AI Act simultaneously.

2.3 Gap 3 — Blind Trust Is Not a Strategy

The Failure Mode

An agent sends an email to a customer. Later, a complaint arrives. Your team asks: what data did the agent use? What decision did it make? Who approved it? The answer is: nobody knows. There is no record. There is no proof. There is nothing to show a regulator, a customer, or a board.

Why auditability is structurally absent:

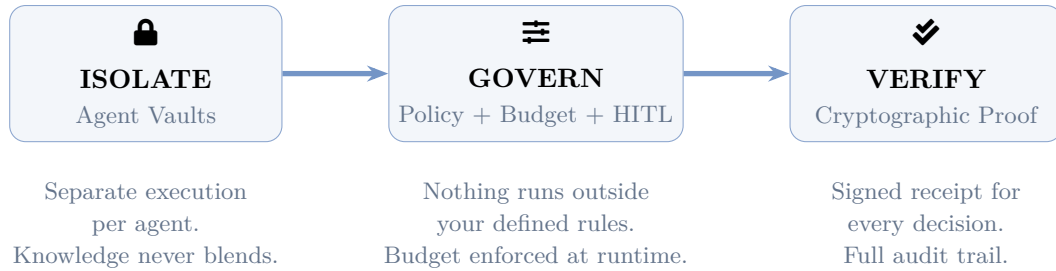
- LLM providers log tokens, not decisions.
- Application logs record API calls, not governance events.
- No current tool produces a cryptographic, third-party-verifiable proof of what an AI agent decided and why.

What Connector does: Every agent execution produces signed, cryptographically-linked receipts across 9 independent chains. The proof bundle is verifiable without access to a live Connector node. A third party — an auditor, a regulator, a court — can verify the bundle using only SHA-256 and Ed25519, the same primitives used to verify Bitcoin transactions.

3 The Architecture: How Connector OS Works

Connector OS is not middleware that wraps LLM calls. It is a control plane that sits between AI agents and the systems they act on — enforcing policy, recording evidence, and producing cryptographic proof.

3.1 The Three-Stage Pipeline



3.1.1 Stage 1 — Isolate (Agent Vaults)

Each agent runs inside a typed namespace sandbox. Its memory, knowledge, tools, and instructions are bound to its namespace. Agents cannot read each other's data. PHI namespaces (/p/) cannot be included in LLM context without a declared capability in the agent's compiled CCL contract. Cross-agent data flow requires an explicit, audited handoff — not a function call.

3.1.2 Stage 2 — Govern (The Safety Net)

The governance pipeline enforces three independent gates before any agent action executes:

1. **Policy Check** — CCL contract declares allowed/denied tools, states, and transitions. The policy engine (Cedar-compatible, deny-overrides) blocks anything undeclared.
2. **Budget Check** — Real token count against the contract's `budget.tokens` ceiling. If exceeded, the contract terminates with a signed receipt — not a crash.
3. **HITL Gate** — Confidence thresholds declared in the CCL contract trigger human-in-the-loop review for high-stakes actions. The approver's identity and decision are recorded in the Compliance Chain.

3.1.3 Stage 3 — Verify (The Proof Plane)

Every governed execution produces entries across 9 independent cryptographic chains:

Chain	Name	What it records
1	Audit Chain	Every system event — HMAC-SHA256 linked
2	Memory Chain	Every namespace write — CID-linked per namespace
3	Dehallucination	Every LLM output linked to grounding memory packets
4	Compliance	Every governance decision tagged with regulation IDs
5	Governance	Every CCL contract step executed
6	Execution	Every tool call receipt — content addressed
7	Trust	Per-agent reputation score history
8	Proof	Root attestation tree anchoring all chains
9	Isolation	Every namespace access — authorized and unauthorized

A single API call — `POST /api/v1/proof/generate` — traverses all 9 chains and returns a JSON proof bundle signed with the node's Ed25519 keypair. The bundle is **verifiable without access to a live Connector node**. This is the property that makes Connector evidence usable in regulatory proceedings.

3.2 CCL Contracts — Policy as Compiled Code

CCL (Connector Contract Language) is a domain-specific language that compiles agent behavior into a *SolutionContract* — a content-addressed, Ed25519-signed, machine-executable governance unit.

CCL Contract Example — Medical Triage Agent

```
contract patient-triage-agent {
  version: "1.0"
  identity: "triage-v1"

  budget {
    tokens:    50000
    cost_usd:  0.80
  }

  tools { allow: [read_ehr, send_alert] deny: [delete, export_data] }
  memory { read:  [/m/triage/, /k/protocols/]
           write: [/m/triage/]           deny_read: [/p/billing/] }

  governance {
    hipaa: true
    hitl_on_confidence_below: 0.75
  }
}
```

The CCL compiler produces a `cls1-sha256-...` CID — a content address that uniquely identifies the compiled contract. Any modification produces a different CID — making

contract tampering detectable.

4 Infrastructure Positioning — Crystal Clear

The One-Sentence Positioning

Connector OS is to AI agents what a Linux kernel is to processes — the mandatory substrate through which all resource access, policy enforcement, and audit recording flows.

4.1 What Connector IS — and What It Is NOT

✓ Connector IS	✗ Connector IS NOT
An execution control plane for AI agents	An AI model or foundation model
A policy enforcement runtime	An orchestration framework (LangChain, CrewAI)
A cryptographic audit and proof layer	A cloud observability tool (Datadog, Grafana)
A compliance-structural layer (HIPAA/-SOC2/GDPR)	A compliance SaaS with checkboxes
An infrastructure primitive (like a kernel)	A wrapper on top of the LLM API
Self-hosted, customer-data-sovereign	A cloud-native SaaS requiring data egress
Model-agnostic (DeepSeek, GPT-4o, Claude)	Coupled to any specific AI provider

4.2 Where Connector Sits in the Stack



Connector sits at the only layer that can simultaneously see: the agent’s declared intent (CCL contract), the data it is about to touch (namespace), the cost it is about to incur (token count), and the system it is about to act on (tool call). No other layer can see all four at once.

4.3 Why LLM Providers Won't Build This

OpenAI, Anthropic, and Google provide LLM inference. Their business model is maximizing token throughput. Building a control plane that blocks, limits, and audits their own API calls is structurally at odds with their revenue model. They also cannot see the customer's data namespaces, CCL contracts, or organizational compliance requirements — these live in the customer's environment.

Evidence: OpenAI's "Usage Limits" are account-level spending caps set by the customer. They are not per-agent, not per-workflow, not cryptographically signed, and not regulation-tagged. They are billing controls, not governance controls.

4.4 Why Orchestration Frameworks Won't Build This

LangChain, CrewAI, AutoGen, and similar frameworks provide agent composition primitives. Their architecture processes requests sequentially and does not include:

- A compiler step that verifies policy before execution begins.
- A content-addressed proof layer that makes audit records tamper-evident.
- A namespace isolation model that classifies data by sensitivity level.
- A cryptographic chain that third parties can verify without trusting the framework vendor.

These are not features these frameworks are missing — they require a different *architectural layer* underneath the framework. Connector is that layer. Frameworks run on top of it.

4.5 Why Cloud Providers Won't Build This

AWS, Azure, and GCP are infrastructure providers. Their AI governance offerings (AWS Bedrock Guardrails, Azure AI Content Safety) operate at the model I/O layer — they filter prompts and responses. They do not:

- Enforce per-agent budget ceilings with real-time token counting.
- Compile agent behavior into a signed, content-addressed contract.
- Produce a regulation-tagged proof bundle covering HIPAA, SOC 2, GDPR, and EU AI Act simultaneously.
- Work across multiple cloud providers and self-hosted models in the same governance plane.

Multi-cloud and model-agnostic governance is structurally outside what a single cloud provider can deliver.

5 Answering the Buyer One-Pager

The buyer one-pager raises six implicit questions. This section answers each with precision.

Q1: What exactly is Connector OS?

Connector OS is an **execution control layer for AI agents**. It is a self-hosted runtime that sits between your AI agents and everything they touch — LLM providers, databases, APIs, and external services. It enforces policy, records cryptographic evidence, and produces compliance proof bundles that satisfy HIPAA, SOC 2, GDPR, and the EU AI Act from a single API call.

Q2: How does the “Silent Budget Drain” get fixed?

Every LLM call flows through the gateway service. Real token counts from the provider’s response are written to a per-agent cost ledger (SQLite, per-agent key). The CCL contract declares a `budget.tokens` ceiling. When the ceiling is hit, the contract state machine moves to `budget_exceeded` — the agent stops, a signed receipt records why, and no further LLM calls are authorized. This happens in the same request-response cycle as the offending call.

Q3: How does the “Compliance Nightmare” get fixed structurally?

Data classification happens at the namespace layer, not the application layer. PHI is stored in `/p/` namespaces with `SecurityLevel::Restricted`. The selective context engine intercepts context construction: any `/p/` content is pseudonymized before the LLM prompt is assembled. Chain 9 records every `/p/` access. The proof bundle shows — cryptographically — that PHI was not included in the LLM context. A regulator does not need to trust your word. They verify the bundle.

Q4: What does “Blind Trust Is Not a Strategy” mean in practice?

Without Connector, when an agent makes a wrong decision, your options are: check application logs (no governance context), ask the LLM provider (they logged tokens, not decisions), or reconstruct from memory (impossible if the agent session has ended). With Connector, every decision produces a cross-chain proof entry. You can call `connectorctl explain <decision-id>` and see: what data the agent read, which CCL step it was on, what the LLM was given, what it returned, whether HITL was triggered, and who approved it — all cryptographically linked.

Q5: What are the three guarantees (Prevent / Enforce / Prove)?

Pillar	Structural mechanism	Evidence produced
Prevent	Namespace firewall blocks PHI from reaching LLM context	Chain 9 entry: <code>phi_in_llm_context: false</code>
Enforce	CCL contract + policy engine + budget gate block every unauthorized action	Chain 5 entry: <code>contract_step, policy_applied</code>
Prove	9-chain proof bundle signed with Ed25519, verifiable offline	<code>proof_root: soe1-sha256-..., verifiable: true</code>

Q6: What does the 30-day pilot produce?

One workflow. Full governance. At the end of 30 days, the pilot customer receives:

- A per-agent cost ledger showing exact LLM spend with provider-verified token counts.
- A regulation-tagged compliance export covering every framework they declared in CCL contracts.
- A chain integrity report showing all 9 chains intact with zero tampered entries.
- A signed proof bundle for each agent run — verifiable without Connector by any auditor.

This is not a demo. This is production evidence. It is what the customer takes to their compliance officer, their board, and their regulator.

6 Moat and Defensibility

The Core Defensibility Claim

Once a regulated enterprise generates compliance proof bundles from Connector, they cannot remove Connector without destroying their audit trail. The moat is compliance lock-in — the same lock-in that made Okta irreplaceable for identity and Palo Alto Networks irreplaceable for network security.

6.1 Technical Moat — The 9-Chain Architecture

The cryptographic chain architecture is not a feature — it is the foundational data structure. Reproducing it requires re-implementing:

1. The CCL compiler (7-pass pipeline: Lex → Parse → Sema → Lower → Optimize → Verify → Emit).
2. The namespace isolation model with typed security levels and chain-linked access records.
3. The proof chain tree that anchors all 9 chains into a Merkle-structured attestation root.
4. The Ed25519 signing infrastructure that makes bundles verifiable without the vendor.

This is 18+ months of foundational engineering that has already been completed. The 9-chain architecture is novel — no existing agent framework, cloud provider, or security tool has published an equivalent design.

6.2 Compiler Moat — CCL as Policy-as-Code

CCL contracts become the authoritative source of truth for what an agent is allowed to do. As enterprises author CCL contracts for their agents, they accumulate a library of governance specifications that describe their AI business logic. These contracts are:

- Machine-verifiable (the compiler catches policy violations at compile time, not at runtime).
- Content-addressed (every version has a CID — tamper detection is automatic).
- Cross-regulation (the same contract satisfies HIPAA, SOC 2, and GDPR simultaneously).

A customer with 50 CCL contracts cannot move those contracts to a competitor without a compiler that understands the CCL semantics. The contracts are the moat.

6.3 Compliance Lock-In

The deepest moat: once a regulated enterprise uses Connector proof bundles in a compliance report, a regulatory filing, or a customer attestation — they cannot retroactively remove Connector without invalidating those records. The chain of custody is established. The tool that produced it becomes infrastructure in the same way that the system of record becomes infrastructure.

6.4 Network Effect

Connector is deployed per-node, and nodes are connected in a cell topology. As a customer deploys more agents:

- More governance data accumulates → adaptive thresholds tighten → security improves.
- The cost ledger produces cross-agent cost attribution → CFO-level visibility emerges.
- The trust chain produces cross-agent reputation data → multi-agent coordination becomes auditable.

The system gets more valuable with every agent added.

7 Why Now — The Regulatory and Market Forcing Functions

7.1 The Regulatory Wave Is Landing

Regulation	Effective	AI-Specific Requirement
EU AI Act (High-Risk AI)	Aug 2026	Risk management system, audit logging, human oversight — mandatory
HIPAA Enforcement Guidance	2025+	AI-generated PHI access requires audit trail same as human access
SEC AI Disclosure Rules	2025	AI-driven decisions in financial context require explainability
SOC 2 Type II	Ongoing	AI systems must satisfy all 5 Trust Service Criteria
NIST AI RMF	Active	Risk identification, measurement, management — enterprise mandated

Every regulation in the table above requires what Connector produces: a tamper-evident record of what the AI did, when, under what policy, with what outcome. The window for voluntary adoption is closing. The window for mandatory adoption is opening.

7.2 The Agentic AI Explosion

Gartner projects that 40% of enterprise applications will embed task-specific AI agents by 2026. OWASP published the first formal taxonomy of agentic AI risks in December 2025, including goal hijacking, tool misuse, identity abuse, memory poisoning, and rogue agents. Microsoft open-sourced the Agent Governance Toolkit in April 2026 — signaling that the problem is now mainstream enough for open-source responses.

These are not indicators of a future market. They are indicators that the market has arrived and governance infrastructure is the immediate next need.

7.3 The Cloud Security Timing Parallel

Greylock has observed that the largest cloud security companies launched years after the cloud migration began. Palo Alto Networks went public in 2012 — six years after AWS launched EC2. Okta launched in 2009 — three years after AWS. CrowdStrike launched in 2011.

The AI agent deployment wave started in 2023–2024. Security and governance infrastructure launched now is exactly on the timing curve that produced those outcomes.

8 GTM — The Entry Wedge and Revenue Architecture

8.1 The Wedge: Regulated Industries with AI in Production

The ICP (Ideal Customer Profile) is **not** every company deploying AI. The wedge is: companies in regulated industries that have already deployed AI agents and now face a compliance examination, a board question about AI risk, or an incident that exposed the absence of an audit trail.

Vertical	Trigger	Regulation	Pilot signal
Healthcare	PHI in AI context	HIPAA, EU AI Act	CISO asking about AI audit logs
Financial Services	AI-driven recommendations	SEC, SOC 2	Compliance team reviewing AI use
Legal / Insurance	AI-generated documents	GDPR, SOC 2	Attorney asking about explainability
Government Contractors	FedRAMP / CMMC	NIST AI RMF	Security review of AI systems

8.2 Revenue Architecture

Tier	Pricing Signal	Value Delivered
Pilot	\$0 / 30 days	One workflow, full governance, production-grade proof bundle
Startup	\$2,000–5,000/mo	Up to 10 agents, full chain, compliance exports
Scale	\$15,000–40,000/mo	Unlimited agents, multi-cell, SLA, HITL dashboard
Enterprise	Custom	Air-gap deploy, dedicated node, custom CCL support, annual audit reports

The pricing model follows the infrastructure pattern: per-node or per-agent subscription, not per-seat. This aligns cost with the value driver (governed AI execution) rather than with organizational headcount.

8.3 The Pilot Is the Sales Motion

The 30-day design partner pilot is not a free trial. It is a **co-production of evidence**. At the end:

- The customer has a compliance proof bundle they can show to an auditor.
- We have a design partner who has proven the product in their specific regulated context.
- The proof bundle is the product — it cannot be generated without Connector.

The conversion argument is not “renew the subscription.” It is “your next audit requires the bundle Connector generates. Removing Connector removes the audit trail.”

9 Competitive Landscape

Category	Examples	Why They Don't Win This
LLM Safety Proxies	Guardrails AI, LLM Guard	Input/output filtering only; no execution governance, no budget, no proof chain
AI Observability	LangSmith, Helicone, Braintrust	Logging and evaluation, not runtime policy enforcement or cryptographic proof
Cloud AI Safety	AWS Bedrock Guardrails, Azure AI Safety	Single-cloud, model I/O layer only; no CCL contracts, no 9-chain audit, no cross-regulation proof
Orchestration	LangChain, CrewAI, AutoGen	Composition layer; no namespace isolation, no budget enforcement, no proof plane
Agent Platforms	Vertex AI Agents, AWS Bedrock Agents	Hyperscaler lock-in; cannot audit across providers; no customer-controlled proof
Connector OS	—	Execution control plane: isolates agents, enforces policy at runtime, produces verifiable compliance proof bundles across all major regulations from a single API call

The category Connector competes in does not yet exist with a named leader. This is the same position Okta was in before identity became a named category, and Palo Alto Networks was in before NGFW became a named category. Greylock invested in both.

10 The Ask — What We Are Looking For from Greylock

The Pilot Offer — For Greylock Portfolio Companies

Connector OS will run one AI workflow, fully governed, for any Greylock portfolio company operating in a regulated industry — at no cost for 30 days.

At the end:

- A production-grade compliance proof bundle for the workflow.
- A full per-agent cost ledger with real token counts.
- A chain integrity report across all 9 chains.
- A CCL contract that encodes the governance policy for that workflow.

If the proof bundle doesn't convince their compliance officer, we have failed. If it does, we have a design partner.

10.1 What We Want to Explore with Greylock

1. **Pattern fit** — Greylock has articulated the AI runtime security problem more precisely than any other investor. We want to understand how Connector maps to their current investment thesis versus what they are actively building conviction around.
2. **Design partner introductions** — Greylock portfolio companies in healthcare AI, fintech, and legal AI are the exact customers who face the three structural gaps we close. A warm introduction reduces our time-to-pilot from weeks to days.
3. **Feedback on the category framing** — We believe “AI execution governance” is the right category name. We want Greylock's perspective on how CISOs and CIOs they speak with would describe this need.

10.2 Why This Is Not a Feature — One Final Time

Greylock's Jason Risch wrote: *“Security controls over AI may themselves also depend on LLMs, adding another non-deterministic circular dependency.”* This is why the control plane cannot be built using LLMs. It must be built in deterministic, formally-verifiable infrastructure. Connector's runtime is written in Rust. The governance engine is a deterministic state machine. The proof chain is HMAC-SHA256 linked data — not an AI model making a judgment.

The system that governs non-deterministic AI must itself be deterministic. That is what Connector OS is.

Run one workflow with us.

30 days • Full traceability • Cost control • Audit-ready proof

GlobalSushrut, Inc. • connector-os.com _____